

The Ever-Evolving Game Engine

How one engine grows with every game — the mechanism, the evidence after nine games, an honest scorecard, and the gaps that remain.

9

games shipped

4

archetypes (genres)

6

automated evals

23

SDK building blocks

The thesis, in one line

The engine evolves by **extraction**: every new game forces a new capability; once a **validator** proves it, it is lifted out of the game and into the SDK as a reusable building block. So the engine is strictly more capable after each game — and the next game starts further ahead.

A platformer became a shooter became a car-RTS became an isometric RTS — on the *same* engine, each genre paying its new muscle back into the shared core.

Every game paid muscle back to the engine

Read the right column as the point: each game didn't just ship — it left the engine more capable for the next one.

GAME	GENRE / FIRST-OF-ITS-KIND	WHAT IT EXTRACTED INTO THE ENGINE (INHERITED BY ALL LATER GAMES)
The Platformer Ember · Nimbus · Deepfin · Jazz	2D platformer	The spine: Studio.Platformer , the deterministic 0-death gate , the Feel fun-model, the level DSL, Materials/Contraptions , Juice , the Shell/Save/Audio chrome, the art + Lyría pipelines, the hub + notes → issues.
Starlance	Vertical space shooter — <i>1st non-platformer</i>	Studio.Shooter (a 2nd archetype) — proof the engine generalises past platformers. The bullet-curtain survivability contract (rules + lint) as a reusable “winnable-by-construction” pattern.
Roadwar	Toon car RTS — <i>1st strategy game</i>	Studio.RTS (3rd archetype) · a buildable economy (Refinery) · the Design Lens (MDA + Schell fun-maximiser) · per-level music · richer Juice (explosions, rings, muzzle, pop-text) · the win-by-construction economic bounds .
Roadwar Iso	Isometric 2.5D RTS — <i>1st 3D-look game</i>	Studio.Iso (a reusable perspective projector for <i>any</i> iso game) · Studio.IsoRTS (4th archetype, a continuous front) · the systematic tension model (rtsPressure) · the Strategy-Sweep + Auto-Balancer evals.

The shape of the growth

Archetypes: 1 → 4

One genre at the start; four today, all on one runtime + one eval contract.

Platformer Shooter RTS IsoRTS

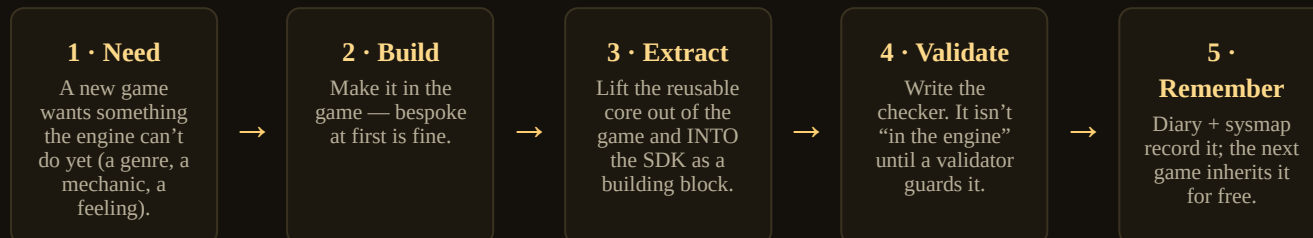
Evals: 1 → 6

The gate was the only judge; now a whole bench grades each game.

gate (0-death) lint feel lens (fun) strategies
balance

How it actually self-evolves

Three forces turn “I shipped a game” into “the engine got better.” Nothing here is automatic magic — it’s a discipline the build follows every time.



The three pillars

① Extraction — games are the engine’s R&D

The engine never grows in the abstract; it grows because a *real game* needed it. Roadwar needed an economy → the Refinery became an SDK system. Roadwar Iso needed an isometric view → **Studio.Iso** became a projector any future iso game uses. The game is the experiment; the building block is the result.

② The validator ratchet — quality only goes up

A capability is only “in the engine” once it has an automated judge: determinism has the **gate**, geometry the **lint**, fun the **Feel** model and the **Design Lens**, challenge the **Strategy-Sweep**, difficulty the **Auto-Balancer**. Because every block ships with its checker, the engine can’t silently regress on that block — the ratchet only turns one way.

③ Memory — the engine records its own growth

The **diary** (35 entries) and the interactive **sysmap** are the engine’s long-term memory: the graph of building blocks, who depends on whom, and why each exists. New work reads this, so the engine compounds instead of repeating itself.

Why this matters: the same loop just produced its sharpest example — the steer “0-death is a weak signal, make it systematic” turned into **three** permanent engine assets (a pressure model, a strategy-sweep eval, an auto-balancer) that every future RTS inherits. The feedback didn’t fix a level; it grew the engine.

An honest scorecard

Mostly yes on the *mechanism*; not yet on *robustness*. The engine demonstrably evolves with every game — but it still leans on discipline where it should lean on automation.

What's genuinely strong

- ▲ **Multi-genre on one core.** Four archetypes share one runtime, one boot, one eval contract, one chrome. The “is it an engine or a platform?” question is answered.
- ▲ **The extraction habit is real.** Every recent game left ≥ 1 reusable block behind (Iso projector, pressure model, the eval tools).
- ▲ **The validator ratchet works.** Fun, challenge, determinism, geometry and difficulty all have automated judges — feedback becomes tooling, not vibes.
- ▲ **Data-only games.** A game is levels + theme; the runtime lives in the SDK, so engine upgrades reach old games.

The real gaps (and how to close them)

△ No regression net

An SDK change can silently break a shipped game; today each gate is run by hand. **Fix:** a CI ratchet that runs every game's gate + lint on any SDK commit — the engine's own test suite. (This session's container reset showed how fragile un-CI'd state is.)

△ Faithfulness by duplication

The pure-Node sims mirror the SDK combat by hand (the gate catches drift). **Fix:** single-source the core so the simulator and the runtime are literally the same code.

△ Genre-locked evals

Strategy-sweep + balance are RTS-only; platformers/shooters have no depth/tension eval. **Fix:** a pluggable “design-pass” interface so each archetype supplies its own depth metric behind one command.

△ Unversioned SDK

The SDK is vendored (copied) per game; copies can drift. **Fix:** semver + a capabilities manifest, so a game declares the engine version it needs and upgrades are intentional.

△ Fragile ship pipeline

Deploys hit CLI crashes + manual workarounds (seen today). **Fix:** bake the known-good deploy path into `studio ship` so it's idempotent and self-healing.

△ Fun model is two-piece

Geometry-Feel + a polish patch, per archetype. **Fix:** one principled fun model that credits design + production uniformly across genres.

The next extraction

Following the engine’s own logic, the highest-leverage next move isn’t a new game — it’s extracting the **CI ratchet** (gaps #1/#2): make the engine guard *all* its games automatically. After that, generalise the design-pass evals so every genre, not just the RTS, gets the fun + depth bench. That is the engine becoming self-defending — the truest sign it’s “ever-evolving.”

Is it ever-evolving? Yes — by design, not by luck

The engine grows because the **build loop forces it to**: a game can't ship without exercising the engine, and the discipline is to **extract + validate + remember** whatever was new. Nine games in, that loop has turned one platformer into a four-genre engine with a six-tool eval bench — and just demonstrated it again, turning a single piece of feedback into three permanent building blocks.

What “done” would look like

- Every SDK commit auto-runs all nine games' gates (the engine defends itself).
- One **studio design-pass** works for every archetype (fun + genre-appropriate depth).
- One simulator core, shared by runtime + evals (no hand-mirroring).
- A versioned SDK with a capabilities manifest; deploys idempotent.

None of these are new genres or art — they're the engine turning its *discipline* into *automation*. That's the last gap between “evolves with every game” (true today) and “can't help but evolve safely with every game” (the goal).

Live proof, today: Roadwar Iso — the 9th game, the 4th genre — shipped at *roadwar-iso-production.up.railway.app*, gate-green and auto-balanced to a rising tension curve, built almost entirely from blocks earlier games left behind.